Rétrospective

Composants

Rétrospective

- ✓ Projet Vue vide
- ✓ Squelette HTML
- ✓ Styles CSS structurels
- ✓ Icônes
- ✓ Routeur pour les pages web
- ✓ Client pour l'API JSON
- ✓ Lecteur audio
- ✓ Local storage pour les favoris
- ✓ Détection online/offline
- ✓ Manifest PWA
- ✓ Caching
- ✓ Service worker

Projet Vite

Rétrospective

- Il est important de se rappeler que Vite est un “packager” qui sert à grouper plusieurs fichiers ensemble
- Il permet également de résoudre des dépendances extérieurs (par exemple, en liant un package installé par npm pour l’utiliser dans l’application)
- Vite fournit un serveur web de dev pour faciliter le développement

Projet Vite

Rétrospective

- Deux commandes principales sont utilisées pour gérer le projet
- `npm install` - Installe toutes les dépendances du projet. S'utilise typiquement à l'installation du projet ou lorsqu'une dépendance est mise à jour
- `npm run start` - Démarre le serveur web de notre projet

Markup HTML et styles CSS

Rétrospective

- Il est important d'utiliser des tags sémantiques variés pour bien différencier les éléments de l'application (privilégier <main>, <section>, <article> à)

- Pour le CSS, privilégier des classes de contrôles à appliquer à un élément, plutôt que de modifier son CSS à la main en javascript
  Par ex: la classe "active" que nous utilisons pour afficher certains éléments

- A noter que ces classes "active" sont manuellement définie dans le CSS que nous utilisons ! Il ne s'agit pas d'une fonction du navigateur !

Créer des composants

Rétrospective

- Dans le squelette, nous avons vu qu'il existe un exemple de markup par page (ou section)
- L'idée est de rendre ce markup dynamique et pour cela, nous devons être capable de le ranger dans des composants
- Le fait que ces composants existent rend leur activation/désactivation plus simple - on ne contrôle plus qu'un seul élément, au lieu d'un ensemble de tags

Créer des composants

Rétrospective

- Créer des composants nous permet également d'encapsuler de la logique en son sein uniquement et pas au niveau global

- Le browser fait exactement pareil avec les éléments HTML. Un tag contient en fait une logique qui lui est propre (exemples : <img>, <audio>, <video>,…)

- Il permet de réutiliser cette structure grâce aux custom elements

Créer des composants

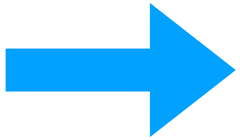
Rétrospective

- Nous avons ensuite choisi, arbitrairement, de créer deux types custom elements :
    - Les “pages” qui affichent une page de l’application et contiennent de la logique d’affichage (chargement, traitement, …)
    - Les “elements” qui ne contiennent que de la logique d’affichage pure

Créer des composants

Rétrospective

```
class BlogPost extends HTMLElement {
   static observedAttributes = ['cover', 'title']
   connectedCallback() {
      this.render()
   }
   attributeChangedCallback() {
      this.render()
   }
   render() {
    this.innerHTML = `
      <img src="${this.getAttribute("cover")}" />
      <div>${this.getAttribute("title")}</div>
     `
   }
}
customElements.define("blog-post", BlogPost)
```



```
<blog-post
  title="Voyager"
  cover="https://…"
>
</blog-post>
```

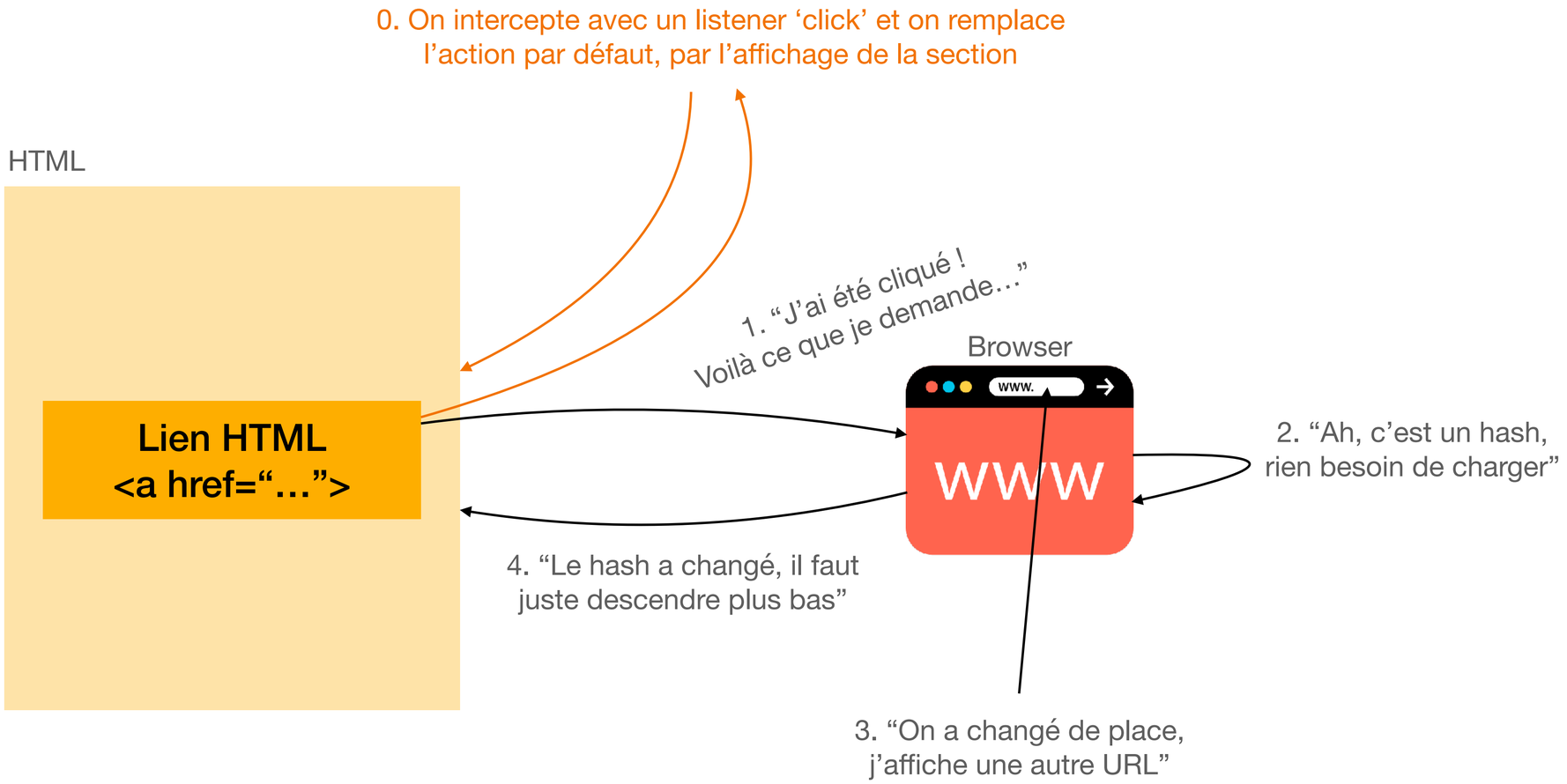
Routeur pour les pages web (changement de section)

Rétrospective

- Un des premiers problème que nous avons rencontré est celui de la gestion des différentes sections - Comment les afficher au bon moment?
- Deux grandes écoles :
  - Mettre des “eventListener” sur des boutons et cacher/afficher (“v1”)
  - Détecter les changements d’URL et réagir en fonction (“v2”)

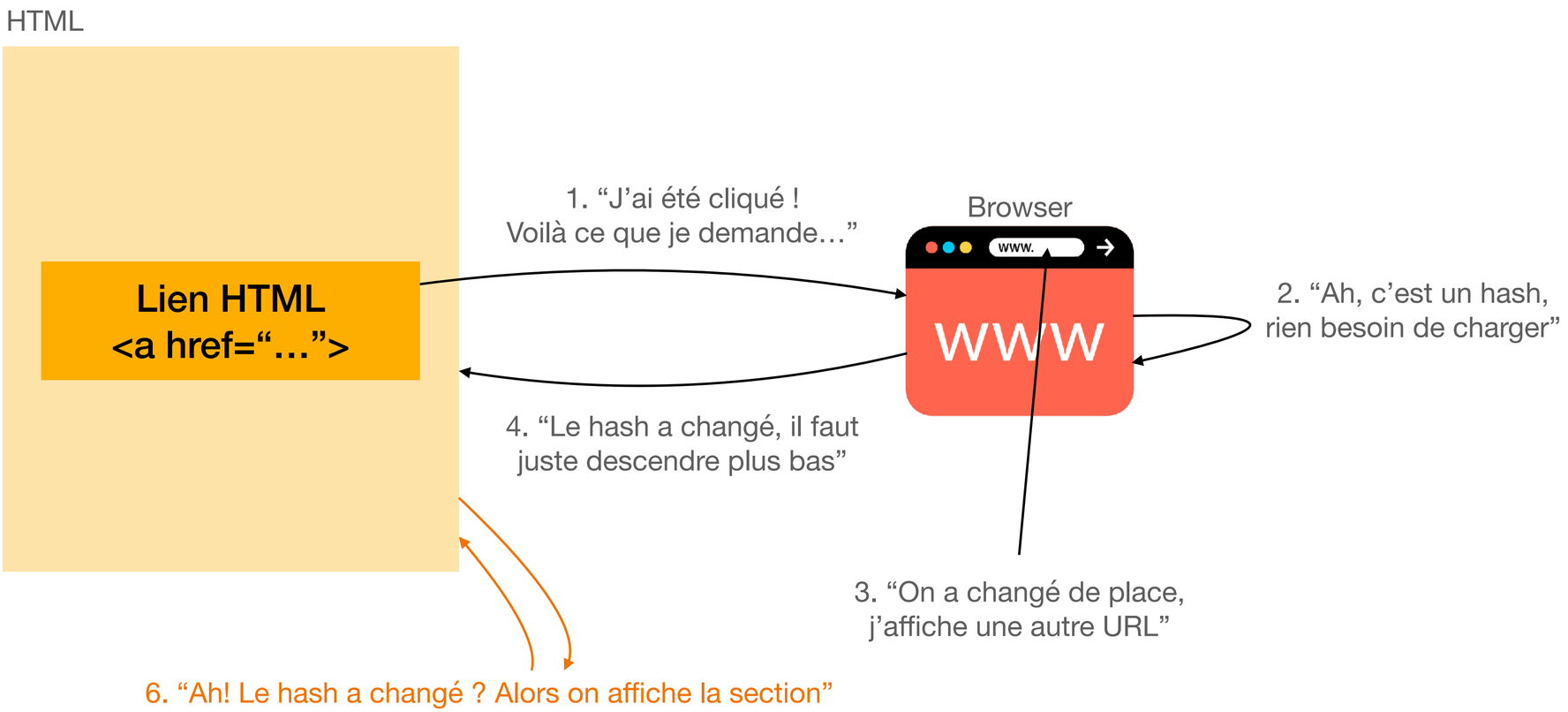
Routeur pour les pages web (changement de section) V1

Rétrospective



Routeur pour les pages web (changement de section) V2

Rétrospective



Routeur pour les pages web (changement de section)

Rétrospective

- Nous nous sommes concentrés sur la v2 - pourquoi ?
  - Le code est moins invasif. Avec un seul eventListener, on peut gérer tous les cas, plutôt que de devoir linker indépendamment chaque élément du menu (et autres liens)
  - Cela permet d'obtenir facilement des URLs différentes par section
    -> Parfait pour les PWA, c'est SEO compliant, on peut référencer des éléments
  - On laisse la fonctionnalité des liens par défaut… Un clic droit "Ouvrir dans un nouvel onglet" fonctionne. Ce n'est pas le cas avec un click listener.

Routeur pour les pages web (changement de section)

Rétrospective

- Comment récupérer l'info ?
- A chaque changement d'URL, le browser émet un événement. Soit "`popstate`" pour tout changement d'URL, soit "`hashchange`" pour un changement dans le hash uniquement
- Comme nous travaillons uniquement avec des hash pour des raisons pratique, "`hashchange`" fait l'affaire

Routeur pour les pages web (changement de section)

Rétrospective

```
const router = () => {
  …
}

window.addEventListener('hashchange', router)
```

Ou aussi….

```
window.addEventListener('hashchange', () => { … })
```

Routeur pour les pages web (changement de section)

Rétrospective

- NB: `hashchange` n'est dispatché que quand l'URL change. Arriver sur une page web avec une URL n'est pas un changement. C'est l'état de base.

- L'idée est donc d'appeler une fois cette fonction au chargement pour que logique s'applique correctement à l'url en cours

Routeur pour les pages web (changement de section)

Rétrospective

```
const router = () => {
  …
}


// On link la fonction "router" à l'événement hashchange pour être
averti d'un changement de hash dans l'url
window.addEventListener('hashchange', router)


// Affichage au chargement pour traiter l'url en cours (exemple: on
ouvre un lien dans un nouvel onglet)
router()
```

Routeur pour les pages web (changement de section)

Rétrospective

- Nous somme maintenant informé d'un changement de hash, c'est à dire lorsque l'utilisateur va cliquer sur un lien du type:

```
<a href="#home">
  …
</a>
```

Routeur pour les pages web (changement de section)

Rétrospective

- Il faut ensuite afficher la bonne section, selon le hash choisi

- Pour cela, il nous suffit de générer le bon HTML dans l'application

- En se basant sur le squelette livré, nous avons décidé de mettre ça dans le tag <main>

Routeur pour les pages web (changement de section)

Rétrospective

- Exemple simplifié

```
const router = () => {
  const hash = window.location.hash

  if(hash == '#artists') {
    document.querySelector('main').innerHTML = 'Les artistes'
    // ou aussi
    document.querySelector('main').innerHTML = '<page-artists />'

  }
}
```

Routeur pour les pages web (changement de section)

Rétrospective

- On souhaite également mettre l'élément du menu actuellement sélectionné en couleur

- On gère cela séparément, car dans certains cas, un même élément du menu est utilisé avec deux sections (typiquement "Musique" est actif pour la liste d'artistes ET la liste des chansons)

Routeur pour les pages web (changement de section)

Rétrospective

- Cela fonctionne bien, mais que faire quand nous avons besoin d'une info présente dans l'URL ? Par exemple l'id d'un artiste ?

- On structure généralement les URLs en mode REST:

  artists —> Liste d'artistes

  artists/12 —> Infos de l'artiste numéro 12

Routeur pour les pages web (changement de section)

Rétrospective

- Comme nous utilisons des hashs, nous pouvons créer une structure similaire aux paths de l'URL, par exemple:

- #artists —> Liste d'artistes

  #artists/12 —> Infos de l'artiste numéro 12

Routeur pour les pages web (changement de section)

Rétrospective

- On peut alors découper l'url en utilisant la fonction split()

- Cela nous donne un tableau qui est structurellement équivalent à notre URL

```
'#artists/12'.split('/') ==> ['#artists', '12']

'#artists'.split('/') ==> ['#artists']
```

Routeur pour les pages web (changement de section)

Rétrospective

- Exemple simplifié

```
const routeur = () => {
  const main = document.querySelector('main')
  const hashs = (window.location.hash || '#home').split('/')

  if(hash == '#artists' && hashs[1]) {
    main.innerHTML = `L'artiste ${hashs[1]}`
  } else if(hash == '#artists' && !hashs[1]) {
    main.innerHTML = 'Les artistes'
  }
}
```

Routeur pour les pages web (changement de section)

Rétrospective

- On sait maintenant gérer des URLs simples et des URLs complexes, avec des paramètres, par exemple

- Plus qu'à rendre cela interactif…!

Client pour l'API Json

Rétrospective

- Notre prochain problème: Charger des données distantes et les transformer en quelque chose d'utilisable en javascript

- Il faut pour cela se référer à l'API Spotlified (voir slides)

- C'est une API JSON. Le contenu doit donc être chargé, puis converti

Client pour l'API Json

Rétrospective

- Problem solved!

```
async function loadJson(url) {
  const response = await fetch(url)
  const parsedJson = await response.json()
  return parsedJson
}
```

Client pour l'API Json - variante promise

Rétrospective

- Problem solved!

```
const loadJson = (url) => fetch(url).then((response) => response.json())
```

Afficher les données

Rétrospective

- Prochain problème: Il nous faut maintenant un moyen de représenter les données chargées dans une liste

- Chaque liste a un markup différent. La liste des artistes n'a pas le même que la liste des chansons

Afficher les données

Rétrospective

- L'idée est de créer une coquille vide d'un élément et de lui passer simplement les informations à afficher via des chaines de caractères (des attributs!).

- Pour cela, nous utilisons un Custom HTML Element qui est spécialement prévu pour ce usecase et permet de ranger le HTML dans un genre de "black box"

- La section "Templating" des slides "Cours 2", pages 56-62 et 78-81 couvrent assez bien ce sujet

Afficher les données

Rétrospective

- Reste ensuite à appliquer ce que nous venons de voir pour chaque élément à afficher

- Typiquement, parcourir le tableau grâce à un forEach et créer un custom Element pour chaque objet du tableau

Afficher les données

Rétrospective

- Nous savons maintenant:
    - Charger des données
    - Les afficher
    - Appeler une logique particulière, basée sur une URL

Afficher les données

Rétrospective

- Prochain problème: interagir avec les données affichées
- Nous savons afficher des données et générer des liens HTML pour afficher d'autres données
- Qu'en est-il de l'interaction ? Par exemple, lire une chanson. Nous avons besoin de plus d'infos que cette simple chanson. Les passer par url semble donc compliqué

Interagir avec les données affichées

Rétrospective

- Chaque élément d'une liste, par exemple, doit pouvoir effectuer une action particulière

- Nous savons que lorsque nous remplissons les informations d'un élément, nous avons accès à tous ses tags HTML

- Nous pouvons nous en servir pour lier des event listener

Interagir avec les données affichées

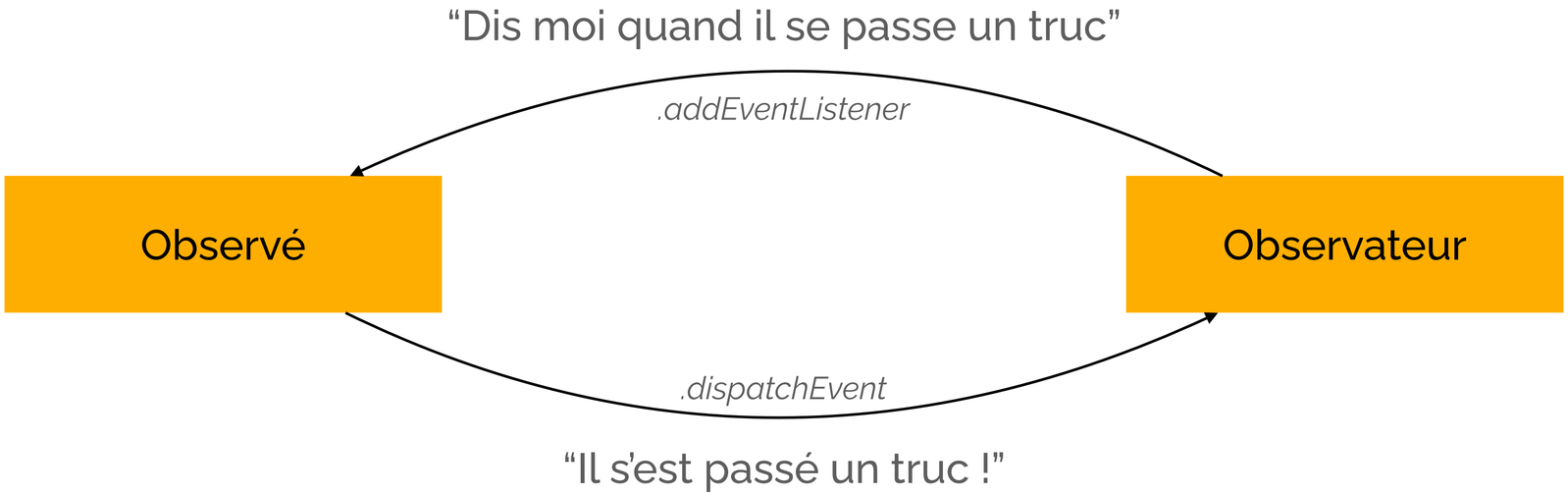
Rétrospective

- On aimerait maintenant être capable d'écouter des événements sur le custom HTML élément que nous avons créé

- Pour garder le concept de l'encapsulation (la black box!), il faudrait une manière d'exposer à l'élément parent (le custom élément) ce qui se passe dans ses enfants, sans que tout le monde soit au courant de la structure interne

Interagir avec les données affichées

Rétrospective



Interagir avec les données affichées

Rétrospective

- A l'image des custom éléments, il existe des custom events !

- Ils permettent de générer ses propres événements et les transmettre à qui est intéressé

Concept

Events, listeners & custom elements

Nom arbitraire

```
<song-item title="Oh My My ft. Séb Mont></song-item>
```

favorite_click play_click

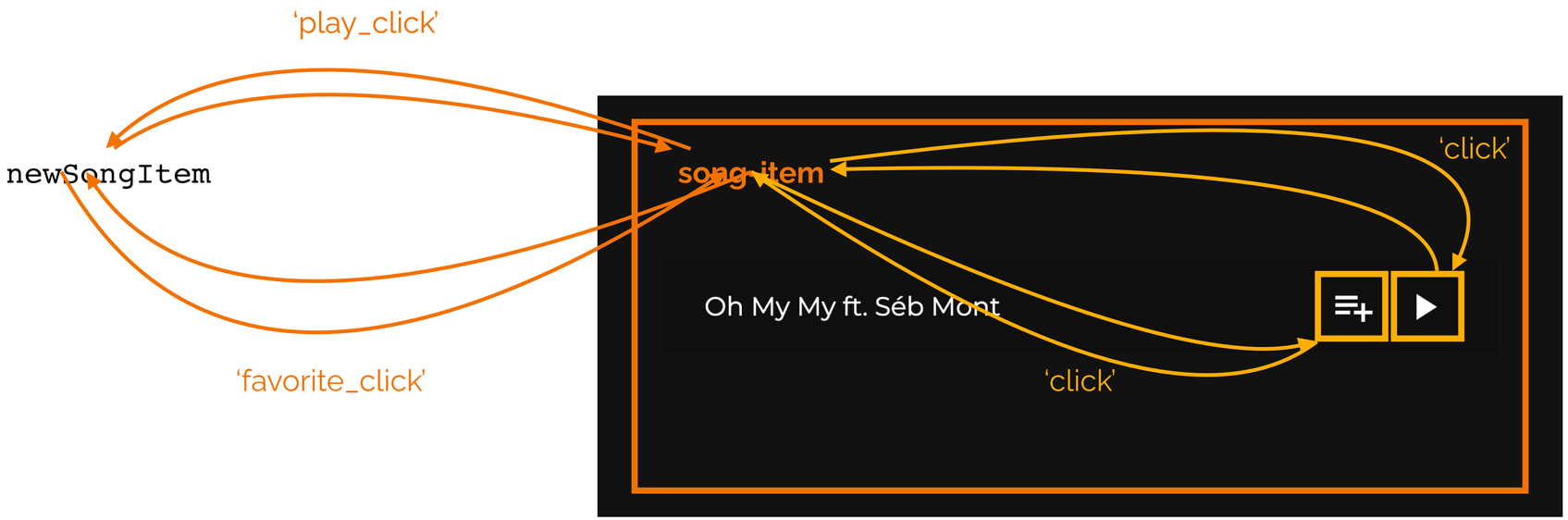
Concept

Events, listeners & custom elements

```
// Déclaration d'un custom event
const playEvent = new CustomEvent('play_click')

class SongItem extends HTMLElement {
   connectedCallback() {
      this.innerHTML = …

      const playButton = this.querySelector('.play-button')

      playButton.addEventListener('click', (e) => {
        e.preventDefault()
        this.dispatchEvent(playEvent)
        // ou
        // this.dispatchEvent(new CustomEvent('play_click'))
      }
   }
}
customElements.define("song-item", SongItem)
```

Concept

Events, listeners & custom elements



Interagir avec les données affichées

Rétrospective

- On peut alors rajouter des events listener

```
const newSongItem = document.createElement('song-item')
// …
newSongItem.addEventListener('play_click', () => console.log('ça play'))
// …
songList.append(newSongItem)
```

- On sait maintenant interagir avec !

Utiliser le player

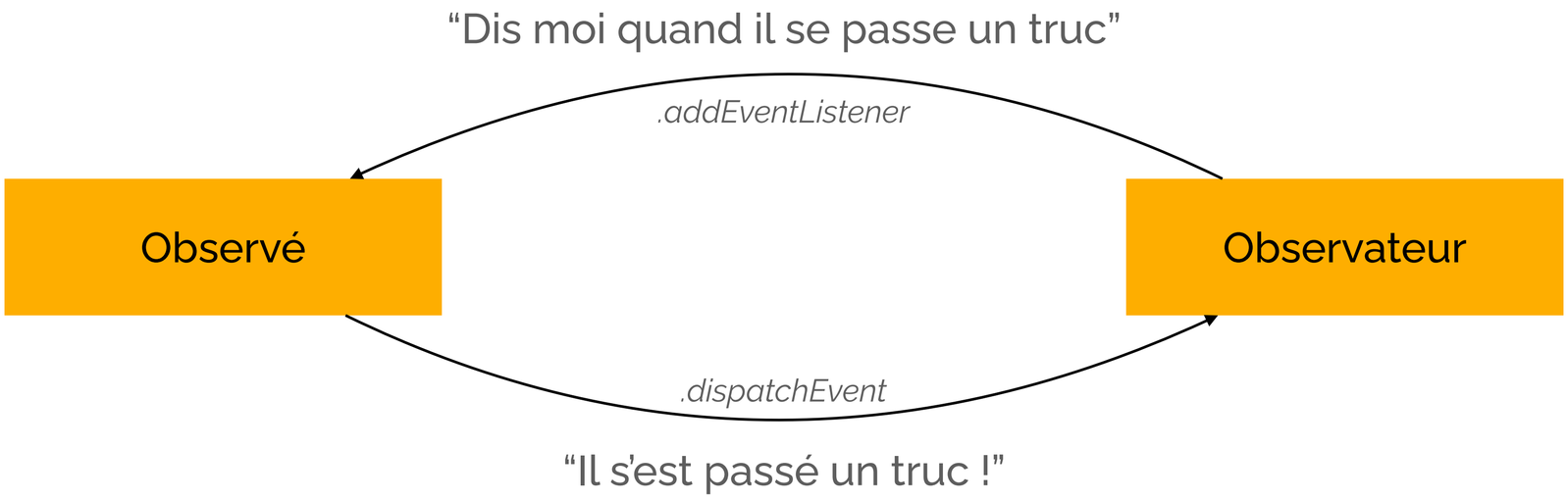
Rétrospective

- Prochain problème: Lire une chanson
- Il y a pour ça plusieurs slides sur le player. Cours 4, pages 1-14
- Cela va surtout consister en des ajouts d'event listeners sur des éléments…
- Il faudra également passer les informations de l'élément cliqué, grâce au listener vu au slide précédent

Utiliser le player

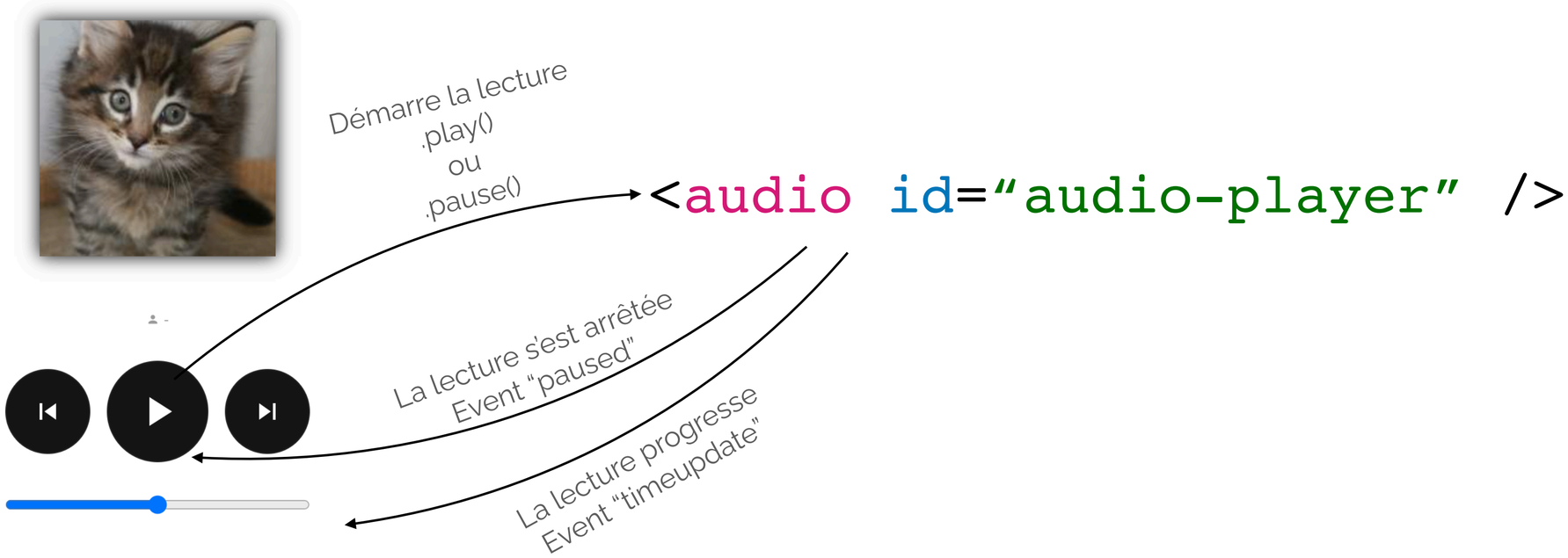
Rétrospective

- Il est important de se rappeler du concept d'observateur/observé



Utiliser le player

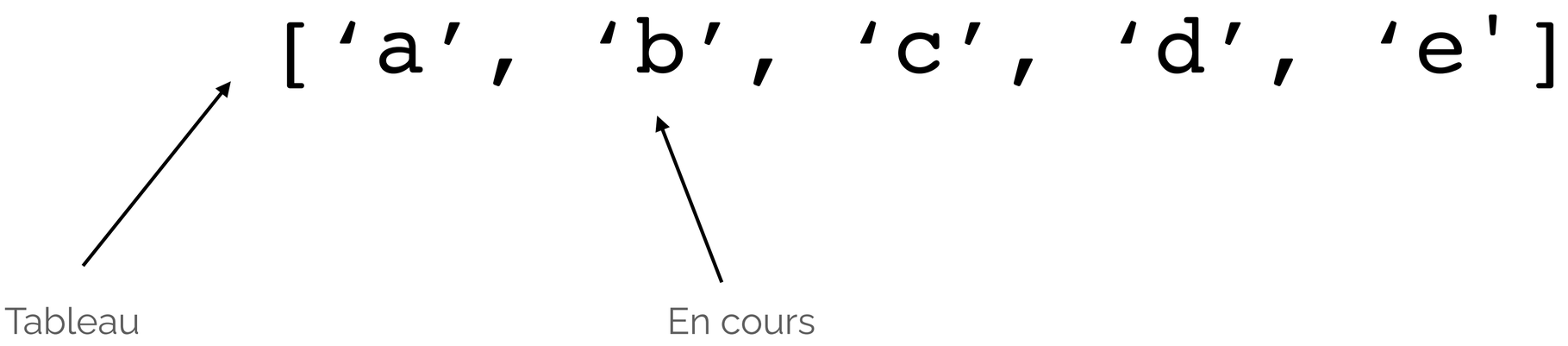
Rétrospective



Utiliser le player

Rétrospective

- Pour la gestion de précédent/suivant, le concept est simple : garder la position d'un élément, au sein d'un tableau d'autres éléments



Utiliser le player

Rétrospective

- Nous avons pour cela créé deux variables que nous modifions via une méthode “playSong”

- La méthode reçoit alors deux infos:
  - La chanson à lire
  - Parmi le tableau transmis

Utiliser le player

Rétrospective

- Lors de son appel, elle s'occupe également de remplacer du contenu HTML d'après les informations reçues (via des modifications d'attributs ou autres variables

Fonction de recherche

Rétrospective

- Prochain problème: Rechercher des chansons

- D'un point de vue abstrait, rechercher des chansons est sensiblement équivalent à charger les chansons d'un artiste:

    - On aimerait une liste de chansons

    - Il y a un critère discriminant - Des chansons qui correspondent au numéro de l'artiste ou qui correspondent à un texte arbitraire

Fonction de recherche

Rétrospective

- Le code devrait donc être sensiblement équivalent…

- On sait maintenant passer des informations dans les URLs et appliquer une méthode particulière, selon ces infos

- Pourquoi ne pas créer une url de recherche '#search' et y passer en paramètre le texte recherché ? '#search/marecherche'

Fonction de recherche

Rétrospective

- Il faut alors utiliser le bon endpoint de l'api pour charger des chansons selon un texte et non un id

- Un moyen de récupérer ces informations sous forme de tableau de chansons, car… nous savons afficher un tableau de chanson !

- Ne reste plus qu'à trouver un moyen de rediriger vers cette url de recherche…

Fonction de recherche

Rétrospective

- L'idée est alors d'écouter les changement sur le champ de recherche et à chaque modification de la valeur, rediriger sur l'url '#search/lavaleur'

```
const searchInput = document.querySelector('#search-input')
searchInput.addEventListener('change', () => {
  window.location.hash = `#search/${encodeURIComponent(searchInput.value)}`
})
```

- Plus qu'à rajouter une condition dans notre hashchange et charger les bonnes chansons

Favoris

Rétrospective

- Prochain problème: Gérer les favoris
- Les favoris ne sont pas gérés par le serveur, mais en local
- Il nous faut donc un genre de tableau, dans lequel stocker des chansons choisies et pouvoir afficher ce tableau
- Egalement un moyen d'ajouter/supprimer des éléments dans ce tableau et vérifier si un élément donné y est présent

Favoris

Rétrospective

- Bonne nouvelle, on sait gérer:
    - Afficher un tableau de chansons
    - Interagir avec les éléments d'une liste
    - Afficher une section, avec une logique particulière
    - Reste juste à trouver un tableau que l'on peut éditer et enregistrer…

Favoris

Rétrospective

- On peut pour cela utiliser JsonStorage

- Il nous faut alors une instance centralisée de celui-ci pour le lire et l'éditer

- localStorage requiert un id pour chaque élément du tableau. Le plus simple est donc d'utiliser l'id de la chanson à ajouter… celui-ci étant unique

Favoris

Rétrospective

- Comme localStorage ne sait pas gérer des objets autres que des chaines, nous avons créé des fonctions d'aide, capable de stocker tout type de contenu grâce à JSON !

```
const setItem = (id, value) => localStorage.setItem(id, JSON.stringify(value))

const getItem = (id) => localStorage.getItem(id) && JSON.parse(localStorage.getItem(id))

const removeItem = (id) => localStorage.removeItem(id)
```

https://developer.mozilla.org/fr/docs/Web/API/Window/localStorage

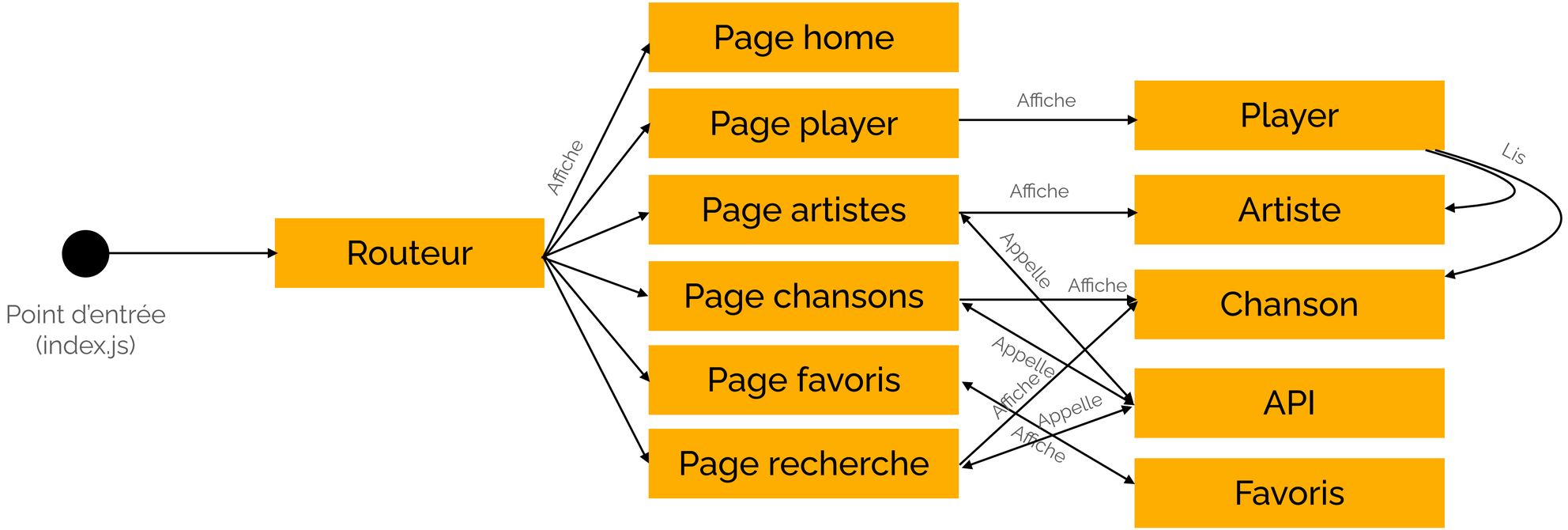
Favoris

Rétrospective

- Ajouter un listener pour le clique sur le bouton “Favori” d’un élément de la liste. Il va ajouter la chanson au tableau des favoris ou l’enlever, selon son état

- Réutiliser l’affichage des chansons, en ne changeant que leur provenance

- Voir solution dans le projet corrigé, ainsi que les slides “Favoris”, Cours 5

Vue globale

Put it together